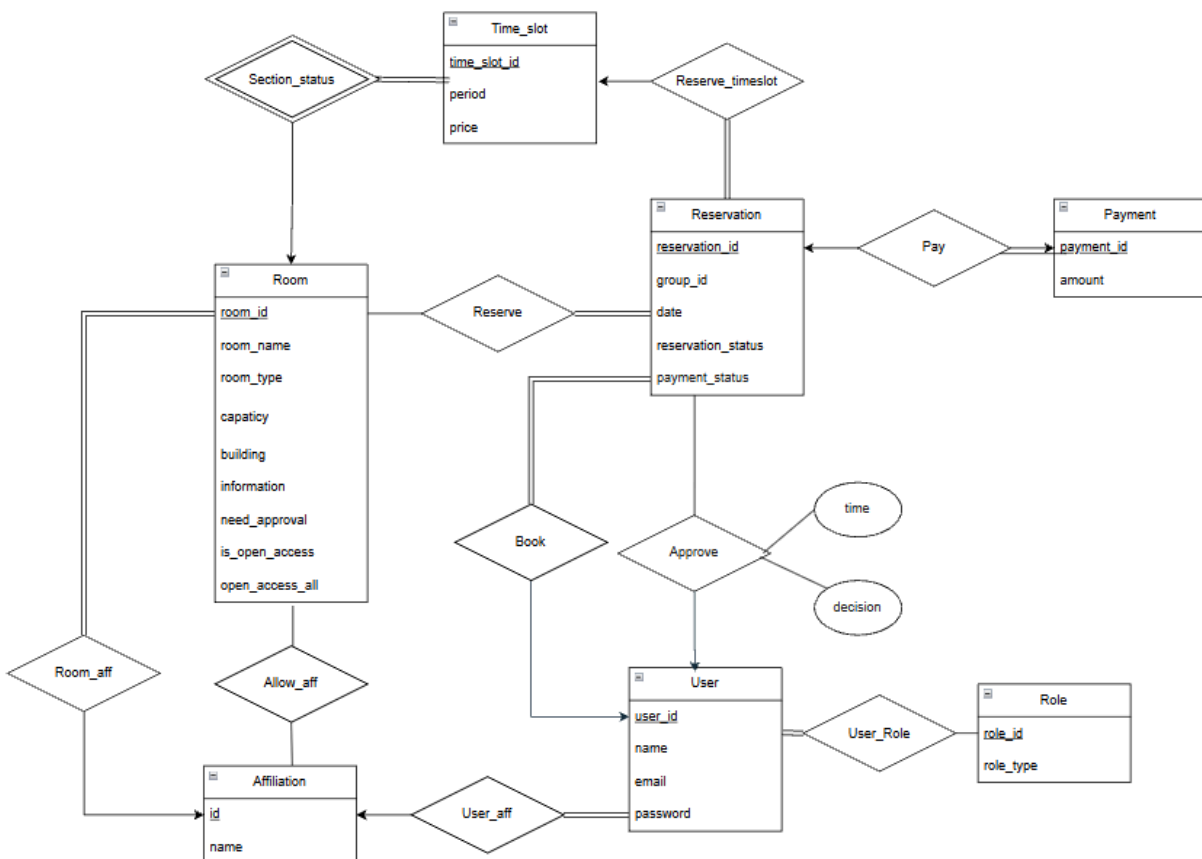


Report3_Team03

41247025S 劉育均 41347064S 張詠勝 41347044S 黃奕嘉 41347036S 余宥騰

Report 2 修改後之 E-R diagram 及 說明



設計圖說明

1. Entity Set

1. User

- 說明
User 用於記錄各個使用者的基本資料，並用user_id進行分別
- Attributes
 - user_id：主鍵
 - name：使用者姓名
 - email：使用者登入電子信箱(帳號)
 - password：使用者登入密碼

2. Role

- 說明
Role 用於區分不同使用者之權限，例如學生、教授、行政人員與管理員。
- Attributes
 - role_id：主鍵
 - role_name：角色名稱

3. Affiliation

- 說明
紀錄所有單位
- Attributes
 - id：主鍵
 - name：單位名稱

4. Room

- 說明
Room 用於描述所有可供預約的空間資訊。
- Attributes
 - room_id：主鍵
 - room_name：空間名稱
 - room_type：空間類型（教室、實驗室、會議室、禮堂、體育館）
 - capacity：容納人數
 - building：建築物
 - information：其他資訊

- need_approval：是否需審核
- open_access：是否開放給其他單位借用
- open_access_all：是否開放給全部單位借用

5. Reservation

- 說明
Reservation 用於記錄使用者對空間的預約申請。
- Attribute
 - reservation_id：主鍵
 - group_id：跨時段預約整合成一筆訂單
 - date：日期
 - reservation_status：預約狀態（待審核、成功、失敗、取消）
 - payment_status: 付款狀態（尚未支付、成功、取消）

6. Payment

- 說明
Payment 用於記錄預約所需繳納金額及繳納狀況
- Attribute
 - payment_id：主鍵
 - amount：需繳納總金額

7. Time_slot (Weak Entity Set)

- 說明
紀錄各時段價錢
- Attribute
 - time_slot_id：主鍵
 - period：時間
 - price：價錢

2. Relationship Set

1. User_Role

- 說明
一位使用者可能同時具有多種角色，而一種角色亦可對應多位使用者。

2. User_aff

- 說明
一位使用者只能具有一個所屬單位，而一個單位可對應多位使用者。

3. Book

- 說明
一位使用者可進行多筆預約，而每筆預約僅屬於一位使用者。

4. Approve

- 說明
一筆預約會由一個使用者（身份為行政人員或管理員）審核並紀錄決定時間和結果。

5. Pay

- 說明
一筆預約至多對應到一筆付款資訊，且每筆付款資訊僅對應到一筆預約。

6. Reserve

- 說明
一個空間可在不同時間被多次預約，但一筆預約可以對應多個空間。

7. Section_status

- 說明
一個Room一定會有每個time_slot的status紀錄。

8. Allow_aff

- 說明
一個room可允許除了自己以外的多個單位借用，一個單位亦可以被允許借用多個其他單位的room。

9. Room_aff

- 說明
一個room一定只會有一個隸屬單位，單位可擁有多個場地。

10. Reserve_timeslot

- 說明
一筆預約只對應一個時段，一個時段可以對應多筆預約。

E-R diagram 轉成 Relational Table 之 DDL 的形式

```
CREATE TABLE affiliation (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100) NOT NULL  
);  
  
CREATE TABLE role (  
    role_id INT PRIMARY KEY AUTO_INCREMENT,  
    role_type VARCHAR(50) NOT NULL  
);  
  
CREATE TABLE `user` (  
    user_id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100) NOT NULL,  
    email VARCHAR(100) NOT NULL UNIQUE,  
    password VARCHAR(255) NOT NULL,  
    affiliation_id INT NOT NULL,  
    FOREIGN KEY (affiliation_id) REFERENCES affiliation(id)  
);  
  
CREATE TABLE user_role (  
    user_id INT NOT NULL,  
    role_id INT NOT NULL,  
    PRIMARY KEY (user_id, role_id),  
    FOREIGN KEY (user_id) REFERENCES `user`(user_id),  
    FOREIGN KEY (role_id) REFERENCES role(role_id)  
);  
  
CREATE TABLE room (  
    room_id INT PRIMARY KEY AUTO_INCREMENT,  
    room_name VARCHAR(100) NOT NULL,  
    room_type VARCHAR(50) NOT NULL,  
    capacity INT NOT NULL,  
    building VARCHAR(100) NOT NULL,
```

```

    affiliation_id INT NOT NULL,
    information TEXT,
    need_approval BOOLEAN DEFAULT FALSE NOT NULL,
    is_open_access BOOLEAN DEFAULT FALSE NOT NULL,
    open_access_all BOOLEAN DEFAULT FALSE NOT NULL,
    FOREIGN KEY (affiliation_id) REFERENCES affiliation(id)
);

CREATE TABLE time_slot (
    room_id INT NOT NULL,
    time_slot_id INT NOT NULL,
    period INT NOT NULL,
    price DECIMAL(10,2) DEFAULT 0 NOT NULL,
    PRIMARY KEY (room_id, time_slot_id),
    FOREIGN KEY (room_id) REFERENCES room(room_id)
);

CREATE TABLE reservation (
    reservation_id INT PRIMARY KEY AUTO_INCREMENT,
    group_id INT NOT NULL,
    user_id INT NOT NULL,
    room_id INT NOT NULL,
    date DATE NOT NULL,
    time_slot_id INT NOT NULL,
    reservation_status VARCHAR(50) NOT NULL,
    payment_status VARCHAR(50),
    FOREIGN KEY (user_id) REFERENCES `user`(user_id),
    FOREIGN KEY (room_id, time_slot_id) REFERENCES time_slot(room_id, time_slot_id)
);

CREATE TABLE payment (
    payment_id INT PRIMARY KEY AUTO_INCREMENT,
    reservation_id INT NOT NULL,
    amount DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (reservation_id) REFERENCES reservation(reservation_id)
);

CREATE TABLE allow_aff (

```

```

    room_id INT NOT NULL,
    affiliation_id INT NOT NULL,
    PRIMARY KEY (room_id, affiliation_id),
    FOREIGN KEY (room_id) REFERENCES room(room_id),
    FOREIGN KEY (affiliation_id) REFERENCES affiliation(id)
);

CREATE TABLE approve (
    reservation_id INT NOT NULL,
    user_id INT NOT NULL,
    time DATETIME NOT NULL,
    decision VARCHAR(50) NOT NULL,
    PRIMARY KEY (reservation_id, user_id),
    FOREIGN KEY (reservation_id) REFERENCES reservation(reservation_
id),
    FOREIGN KEY (user_id) REFERENCES `user`(user_id)
);

```

系統內容功能詳細敘述

本系統為「校園空間租借系統」，主要目的在於整合校園內各類空間之查詢、預約、審核、付款與紀錄管理功能。透過資料庫統一管理使用者、角色、單位、空間、時段、預約、審核與付款資料，降低人工處理造成的資料不一致、重複預約與管理困難等問題。

系統主要使用者包含學生、教授、行政人員與系統管理員。不同角色具有不同操作權限，例如學生與教授主要進行空間查詢與預約，行政人員可進行審核與費用處理，系統管理員則負責系統資料維護與權限設定。本系統透過 `user`、`role`、`user_role` 與 `affiliation` 等資料表，記錄使用者基本資料、所屬單位與角色權限，作為後續功能控管的依據。

一、使用者與權限管理功能

使用者管理功能主要用於記錄系統中所有使用者的基本資料，包括姓名、電子信箱、密碼與所屬單位。每位使用者可以具有一種或多種角色，例如同一位使用者可能同時是教授與行政人員，因此系統以 `user_role` 關聯表處理使用者與角色之間的多對多關係。

系統會依照使用者角色決定其可使用的功能。學生可查詢空間資訊並預約一般教室或會議室；教授可預約教室、會議室與實驗室；行政人員除了可預約空間外，也可審核需要核准的預約申請；系統管理員則可維護空間資料、角色資料與系統設定。透過此功能，可避免未授權使用者操作高階管理功能，確保系統使用安全性與資料正確性。

二、單位管理功能

單位管理功能用於記錄校內不同系所或行政單位，例如資工系、數學系、總務處等。每位使用者會隸屬於一個單位，而每個空間也會隸屬於一個管理單位。此資料主要透過 `affiliation` 表管理，並由 `user.affiliation_id` 與 `room.affiliation_id` 建立關聯。

單位資料會影響空間借用權限。例如某些空間只允許同單位使用者借用；某些空間可開放給特定其他單位借用；也有部分空間可開放給全部單位使用。因此，單位管理功能是判斷使用者是否具有借用資格的重要依據。

三、空間資料管理功能

空間資料管理功能用於建立與維護校園內可供預約的空間資料。每個空間會記錄空間名稱、空間類型、容納人數、所在建築物、所屬單位、其他資訊、是否需要審核、是否開放給其他單位借用，以及是否開放給所有單位借用。

空間類型可包含教室、實驗室、會議室、禮堂與體育館等。系統管理員可新增、修改或刪除空間資料。例如新增一間會議室時，需要設定其容量、建築物、所屬單位、是否需要審核，以及各時段的價格。

此功能對應資料表主要包含 `room`、`time_slot` 與 `allow_aff`。其中 `room` 儲存空間基本資料，`time_slot` 儲存該空間不同時段與價格，`allow_aff` 則記錄該空間額外允許哪些單位借用。

四、空間查詢功能

空間查詢功能提供使用者查詢校園內可借用空間的資訊。使用者可依照空間名稱、空間類型、建築物、容納人數、所屬單位或開放狀態進行搜尋。查詢結果會顯示空間基本資料，例如空間名稱、類型、容量、位置、所屬單位、補充說明，以及該空間是否需要審核。

此外，系統也會依照日期與時段顯示該空間目前的使用狀態。若某空間在指定日期與時段已有預約成功紀錄，則該時段會顯示為已預約；若沒有成功預約紀錄，則顯示為可預約。根據系統規則，只有預約成功的紀錄會影響可用狀態，待審核或預約失敗的申請不應直接佔用時段。

此功能可協助使用者快速找到符合需求且可使用的空間，並降低人工詢問與重複查詢的成本。

五、空間預約申請功能

空間預約功能是本系統的核心功能。使用者可選擇欲借用的空間、日期與時段，送出預約申請。系統會在建立預約前檢查使用者是否符合借用資格，包括角色權限、所屬單位權限、空間開放設定，以及該時段是否已被成功預約。

每筆預約資料會記錄於 `reservation` 表中，包含預約編號、群組編號、申請者、空間、日期、時段、預約狀態與付款狀態。其中 `group_id` 可用於整合跨時段預約，例如使用者一次預約多個空間連續多個時段時，可將多筆 `reservation` 視為同一組訂單，方便後續查詢與付款。

預約狀態可分為待審核、成功、失敗與取消。若該空間不需要審核，且預約時段沒有衝突，系統可直接將預約狀態設定為成功。若該空間需要審核，則系統會先將預約狀態設定為待審核，等待行政人員或管理員進行審核。

六、預約衝突檢查功能

為避免同一空間在同一日期與同一時段被重複預約，系統在使用者送出預約申請時，必須進行衝突檢查。系統會根據 `room_id`、`date` 與 `time_slot_id` 查詢是否已有相同條件且狀態為成功的預約紀錄。

若該時段已存在成功預約，系統應拒絕新的預約申請，或提示該時段已被借用。若該空間需要審核，同一使用者不可重複送出同一空間、同一日期、同一時段的待審核申請，但不同使用者仍可送出申請，由審核者最後決定通過哪一筆；一旦某筆申請審核通過，其他相同時段的待審核申請自動標記為失敗。

七、預約審核功能

部分空間可能需要經過行政人員審核後才能使用，例如總務處管理的場地、跨單位借用的場地，或較特殊的實驗室與禮堂。當使用者申請這類空間時，系統會先建立待審核預約，並等待具有審核權限的使用者處理。

審核資料會記錄於 `approve` 表中，包含預約編號、審核者、審核時間與審核結果。審核者可以選擇通過或拒絕申請。若審核通過，系統會將該筆預約狀態更新為成功；若審核拒絕，則更新為失敗。

在審核通過前，該時段對一般查詢可視為尚未被正式占用。當某一筆預約成功後，系統應確保相同空間、日期與時段不會再有其他成功預約。若存在其他待審核申請，系統自動將其標記為失敗。

八、付款與費用計算功能

付款與費用計算功能用於處理需要收費的空間租借。系統會根據使用者預約的空間、時段與各時段價格計算應繳金額。每個空間的不同時段價格記錄於 `time_slot` 表中的 `price` 欄位。

費用計算方式可依照規則設定為使用時間 × 時段價格

若使用者預約多個連續時段，系統會將所有時段的價格加總，得到該次預約或該組訂單的總金額。付款資料記錄於 `payment` 表中，包含付款編號、對應預約與金額。預約表中的 `payment_status` 則記錄付款狀態，例如尚未支付、成功或取消。

若場地屬於總務處或系統設定為需收費場地，無論使用者身分皆需付款。若場地不需收費，則不會紀錄於 `payment` 表中。

九、預約取消功能

使用者可取消尚未開始的預約。當使用者取消預約後，系統會將該筆預約的 `reservation_status` 更新為取消，並立即釋出該空間在該日期與時段的使用權，使其他使用者可以重新預約該時段。

若該預約已產生付款資料，系統也需要同步更新付款狀態。例如尚未付款的預約取消後，付款狀態可更新為取消；若已付款，則進行人工確認取消。

十、使用紀錄查詢功能

系統需保留所有預約、審核與付款紀錄，供使用者與管理者查詢。一般使用者可查詢自己的預約紀錄，包括預約空間、日期、時段、狀態與付款狀態。行政人員與系統管理員則可查詢所有使用者的預約紀錄，並可依照日期、空間、單位或預約狀態進行篩選。

此功能可協助學校追蹤空間使用情形，例如哪些空間最常被預約、哪些時段使用率較高、哪些申請被取消或審核失敗。這些資料可作為未來空間配置、管理政策調整與設備改善的依據。

十一、開放單位管理功能

由於不同空間可能有不同借用限制，因此系統需要管理空間是否開放給其他單位借用。每個空間本身會有一個所屬單位，記錄於 `room.affiliation_id`。若該空間只開放給所屬單位使用，則其他單位使用者不可預約。

若 `is_open_access` 為 true，表示該空間可開放給特定其他單位使用，允許借用的單位會記錄在 `allow_aff` 表中。若 `open_access_all` 為 true，表示該空間開放給所有單位使用，不需要逐一設定允許單位。

當使用者預約空間時，系統會根據使用者的 `affiliation_id`、空間的所屬單位、`is_open_access`、`open_access_all` 與 `allow_aff` 資料判斷是否允許預約。此功能可讓不同單位彈性管理自己的空間，同時也能支援跨單位借用需求。

十二、時段與價格管理功能

每個空間會有對應的時段資料，記錄於 `time_slot` 表中。由於不同空間可能有不同的可借用時段與價格，因此 `time_slot` 採用 `(room_id, time_slot_id)` 作為複合主鍵，表示每個空間可以擁有自己的時段設定。

時段資料包含時段編號、時間區段與價格。系統可依照這些資料顯示空間的可借用時間，並在使用者預約時進行費用計算。例如某會議室上午8:00~9:00時段價格為 500 元，下午14:00~15:00時段價格為 800 元，系統可依照使用者選擇的時段自動加總金額。

此設計可支援不同空間具有不同收費規則，也方便未來擴充更多時段或價格設定。

十三、系統管理功能

系統管理員負責維護整個系統的基礎資料與設定，包括使用者資料、單位資料、空間資料、開放單位資料、時段價格資料等。管理員也可檢視所有預約、審核與付款紀錄，確保系統資料正確。

透過系統管理功能，可提升整體系統的彈性與可維護性。

十四、資料一致性與限制規則

為確保系統資料正確，資料庫需透過主鍵、外鍵與關聯表維持資料一致性。例如使用者必須隸屬於有效的單位，預約必須對應到有效的使用者、空間與時段，付款必須對應到有效的預約，審核紀錄也必須對應到有效的預約與審核者。

系統也應限制同一空間在同一日期與同一時段只能有一筆成功預約，避免重複借用。若使用者取消預約，該時段應立即釋出。若空間需要審核，則必須經過審核通過後才會成為正式預約。

透過上述限制，系統可滿足多使用者同時查詢與預約的需求，並降低資料錯誤、重複預約與權限混亂的情況。

系統功能實作規劃

本系統採用前後端分離架構進行開發，前端負責提供使用者操作介面，後端負責處理業務邏輯、權限判斷、資料驗證與資料庫存取。整體系統以 Laravel 作為後端框架，Vue.js 作為前端框架，MySQL 作為資料庫管理系統。使用者可透過網頁介面進行登入、查詢空間、送出預約、查看預約紀錄；行政人員則可進行審核付款；系統管理員則可空間管理、時段價格管理與資料維護等。

本系統的主要功能實作規劃可分為：使用者登入與權限控管、空間查詢、空間預約、預約衝突檢查、預約審核、付款與費用計算、使用紀錄查詢、空間與時段管理等部分。

一、系統整體流程規劃

系統整體流程由使用者登入開始，系統會依照使用者的角色與所屬單位決定可操作的功能。一般使用者登入後，可進入空間查詢頁面，會顯示可借用的空間、日期與時段。當使用者送出預約申請後，後端會檢查該空間在指定日期與時段是否已有成功預約。

若空間不需要審核，且沒有時段衝突，系統會直接建立成功預約紀錄。若空間需要審核，系統會先建立待審核預約紀錄，等待行政人員進行審核。行政人員登入後，可查看待審核申請，並決定通過或拒絕。若通過，系統會將預約狀態改為成功；若拒絕，則改為失敗。

若該空間需要付款，系統會依照使用者選擇的時段與價格計算總金額，並建立付款資料。使用者可在預約紀錄頁面查看預約狀態與付款狀態。若使用者取消尚未開始的預約，系統會將預約狀態改為取消，並釋出該時段供其他使用者預約。

二、主要功能流程圖規劃

1. 使用者登入流程

使用者進入系統後，需輸入電子信箱與密碼。系統會將使用者輸入的帳號資料送至後端，由後端查詢 `user` 資料表確認帳號是否存在，並驗證密碼是否正確。驗證成功後，系統會讀取該使用者的角色與所屬單位，並依照角色導向不同的操作頁面。

流程如下：

```
graph TD; A[進入登入頁面] --> B[輸入 email 與 password]; B --> C[後端驗證帳號密碼]; C --> D[驗證成功 驗證失敗 → 顯示錯誤訊息]; D --> E[讀取 user_role 與 affiliation 資料]; E --> F[依角色導向對應 Dashboard];
```

學生與教授登入後主要進入空間查詢與預約頁面；行政人員登入後可查看待審核申請和空間查詢與預約頁面；系統管理員登入後可進入後台管理頁面。

2. 空間查詢流程

使用者可依照空間類型、建築物、關鍵字、日期查詢可借用的空間。前端會將查詢條件送至後端，後端根據 `room`、`time_slot`、`reservation` 等資料表進行查詢。

系統查詢時會判斷該空間在指定日期與時段是否已有成功預約。若已有成功預約，則該時段顯示為已預約；若沒有成功預約，則顯示為可預約。

流程如下：

```
graph TD; A[使用者進入空間查詢頁面] --> B[選擇查詢條件  
例如：空間類型、建築物、日期、時段]; B --> C[前端送出查詢請求]; C --> D[後端查詢 room、time_slot、reservation]; D --> E[判斷該時段是否已有成功預約]; E --> F[回傳空間列表與可用狀態]; F --> G[前端顯示查詢結果];
```

3. 空間預約流程

使用者在查詢結果中選擇欲借用的空間與時段後，可送出預約申請。後端會先檢查使用者是否符合借用資格，例如是否為同單位、是否為開放給全部單位、是否存在於允許借用單位清單中。

接著系統會進行時段衝突檢查，確認同一空間、同一日期、同一時段是否已有成功預約。若沒有衝突，系統會依照空間是否需要審核決定預約狀態。

流程如下：

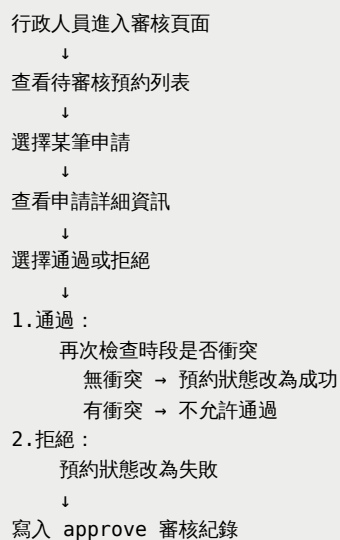


4. 預約審核流程

當使用者預約需要審核的空間時，系統會建立待審核紀錄。行政人員登入後，可在審核頁面查看所有待審核申請，包含申請人、空間名稱、日期、時段、申請單位與是否需付款等資訊。

行政人員可選擇通過或拒絕申請。通過時，系統會再次檢查該時段是否已有其他成功預約，避免在等待審核期間發生衝突。若沒有衝突，系統會將預約狀態改為成功，並在 **approve** 表中記錄審核者、審核時間與審核結果。若拒絕，則將預約狀態改為失敗。

流程如下：



5. 付款與費用計算流程

若預約的空間需要收費，系統會根據使用者選擇的時段計算總金額。每個空間的時段價格儲存在 `time_slot.price` 欄位中。若使用者一次選擇多個時段，系統會將各時段價格加總，計算出總金額。

付款資訊會儲存在 `payment` 表中，並透過 `reservation.payment_status` 記錄付款狀態。若空間不需收費，則不建立付款資料，或將付款狀態設定為不需付款。

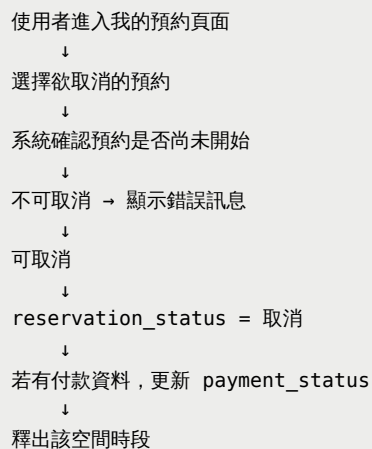
流程如下：



6. 取消預約流程

使用者可在自己的預約紀錄頁面取消尚未開始的預約。系統會先確認該預約日期與時段是否尚未開始。若符合條件，系統會將 `reservation_status` 更新為取消，並釋出該時段。

流程如下：



三、介面設計規劃

本系統介面可依照使用者角色分為一般使用者介面、行政人員介面與系統管理員介面。

1. 登入頁面

登入頁面提供使用者輸入電子信箱與密碼。登入成功後，系統會依照使用者角色導向不同頁面。

主要欄位包含：

Email
Password
登入按鈕

若帳號或密碼錯誤，系統會顯示「帳號或密碼錯誤」。若登入成功，則依角色進入對應 Dashboard。

2. 一般使用者 Dashboard

學生與教授登入後會進入一般使用者 Dashboard。此頁面主要提供快速入口，例如空間查詢、預約查詢、我的預約。

主要功能區塊包含：

空間查詢
預約查詢
我的預約

學生與教授的差異在於可預約的空間類型不同。學生主要可預約一般教室或會議室；教授可預約教室、會議室與實驗室。

3. 空間查詢頁面

空間查詢頁面是使用者最常使用的頁面。使用者可透過篩選條件查找適合的空間。

主要篩選條件包含：

空間名稱關鍵字
空間類型
建築物
設備資訊

查詢結果以列表或卡片方式呈現，每筆資料包含：

空間名稱
空間類型
建築物

容納人數
所屬單位
設備資訊
是否需要審核

4. 空間詳細資訊頁面

使用者點選某個空間後，可進入空間詳細資訊頁面。此頁面會顯示該空間完整資料與不同時段的價格。

主要內容包含：

空間名稱
空間類型
建築物
容納人數
所屬單位
空間說明
是否需要審核
可預約時段
費率

此頁面可讓使用者確認空間資訊後再送出預約，降低誤選空間或時段的情況。

5. 預約申請頁面

預約申請頁面讓使用者確認預約資訊。使用者送出前，系統會顯示空間名稱、日期、時段與是否需要審核。

主要內容包含：

空間名稱
日期
時段
金額
是否需要審核
送出申請按鈕

使用者按下送出後，前端會呼叫後端 API 建立預約。若成功，系統顯示預約成功或已送出審核；若失敗，則顯示失敗原因，例如時段已被預約。

6. 我的預約頁面

我的預約頁面提供使用者查看自己的預約紀錄。使用者可查看預約狀態、付款狀態，並取消尚未開始的預約。

主要欄位包含：

空間名稱
日期
時段
預約狀態
付款狀態
金額
操作按鈕

操作按鈕可包含：

查看詳細資料
取消預約

若預約已開始或已取消，則取消按鈕不可使用。

7. 行政人員審核頁面

行政人員可在審核頁面查看所有待審核預約。此頁面會顯示申請人、空間、日期、時段與申請狀態。

主要欄位包含

申請人
空間名稱
日期
時段
申請狀態
操作按鈕

操作按鈕包含：

查看詳細資料
通過
拒絕

當行政人員點選通過或拒絕後，系統會將審核結果寫入 `approve` 表，並更新 `reservation.reservation_status`。

8. 行政人員付款管理頁面

行政人員可在付款管理頁面查看所有需付款的預約。此頁面會顯示申請人、空間、日期、時段與申請狀態。

主要欄位包含

申請人
空間名稱
日期
時段
總金額
付款狀態
操作按鈕

操作按鈕包含：

標記已付/未付

當行政人員點選標記已付或標記未付後，系統會將結果寫入 `reservation` 表，並更新 `reservation.payment_status`。

9. 系統管理員後台頁面

系統管理員後台主要負責維護系統基礎資料，包括使用者、單位、空間、時段與開放單位設定。

主要功能包含：

使用者管理
單位管理
空間管理(包含時段價格、開放單位)
預約紀錄查詢
付款紀錄查詢

在空間管理頁面中，管理員可新增、修改或刪除空間資料。新增空間時，需要設定空間名稱、類型、容量、建築物、所屬單位、是否需要審核、是否開放借用與時段價格。

四、前端與後端連接規劃

本系統前端使用 Vue.js 開發，負責畫面呈現與使用者操作。後端使用 Laravel 開發，負責 API、資料驗證、權限控管、邏輯與資料庫存取。

前端會透過 HTTP Request 呼叫後端 API，例如登入、查詢空間、建立預約、取消預約與審核預約。後端接收到請求後，會透過 Controller 處理邏輯，再透過 Model 與 MySQL 資料庫互動。

整體連接方式如下：

```
Vue 前端頁面
  ↓ HTTP Request
Laravel API Route
  ↓
Controller
```

```
↓  
Service / 驗證邏輯  
↓  
Model  
↓  
MySQL Database  
↓  
回傳 JSON 給前端  
↓  
Vue 更新畫面
```

五、後端 Laravel 架構規劃

Laravel 後端可依照資料表建立對應 Model，並建立 Controller 處理各功能。

1. Model 規劃

```
User.php  
Role.php  
Affiliation.php  
Room.php  
TimeSlot.php  
Reservation.php  
Payment.php  
Approve.php  
AllowAff.php
```

各 Model 對應資料庫資料表，用於查詢、新增、修改與刪除資料。

例如：

```
User 對應 user 資料表  
Room 對應 room 資料表  
Reservation 對應 reservation 資料表  
Payment 對應 payment 資料表  
Approve 對應 approve 資料表
```

2. Controller 規劃

```
AuthController.php  
RoomController.php  
ReservationController.php  
ApprovalController.php  
PaymentController.php  
UserController.php  
AffiliationController.php  
TimeSlotController.php
```

各 Controller 負責處理不同功能：

AuthController：登入、登出、取得目前使用者資料
RoomController：空間查詢、空間詳細資料、新增空間、修改空間
ReservationController：建立預約、取消預約、查詢預約紀錄
ApprovalController：查詢待審核預約、通過預約、拒絕預約
PaymentController：計算費用、建立付款紀錄、更新付款狀態
UserController：使用者資料管理
AffiliationController：單位資料管理
TimeSlotController：時段與價格管理

3. API Route 規劃

後端可設計以下 API：

```
POST    /api/login
POST    /api/logout
GET     /api/user

GET     /api/rooms
GET     /api/rooms/{id}
GET     /api/rooms/{room}/available-time-slots

GET     /api/time-slots
POST    /api/time-slots
PUT     /api/time-slots/{id}

POST    /api/reservations
GET     /api/reservations/my
POST    /api/reservations
GET     /api/reservations/{reservation}
PATCH  /api/reservations/{reservation}/cancel
PATCH  /api/reservation-groups/{group_id}/cancel

GET     /api/approvals/pending
GET     /api/approvals/history
PATCH  /api/approvals/{reservation}/approve
PATCH  /api/approvals/{reservation}/reject

GET     /api/payments/my
GET     /api/payments/{payment}
PATCH  /api/payments/{payment}/confirm
PATCH  /api/payments/{payment}/cancel

GET     /api/admin/rooms
POST    /api/admin/rooms
PUT     /api/admin/rooms/{room}
DELETE  /api/admin/rooms/{room}

GET     /api/admin/rooms/{room}/time-slots
POST    /api/admin/rooms/{room}/time-slots
PUT     /api/admin/rooms/{room}/time-slots/{time_slot}
DELETE  /api/admin/rooms/{room}/time-slots/{time_slot}
```

```

PATCH /api/admin/rooms/{room}/time-slots/{time_slot}/status

GET /api/admin/users
POST /api/admin/users
PUT /api/admin/users/{user}
DELETE /api/admin/users/{user}
PUT /api/admin/users/{user}/roles
PATCH /api/admin/users/{user}/status

GET /api/admin/affiliations
POST /api/admin/affiliations
PUT /api/admin/affiliations/{affiliation}
DELETE /api/admin/affiliations/{affiliation}

GET /api/admin/rooms/{room}/allowed-affiliations
POST /api/admin/rooms/{room}/allowed-affiliations
DELETE /api/admin/rooms/{room}/allowed-affiliations/{affiliation}

GET /api/admin/reports/reservations
GET /api/admin/reports/reservations/monthly
GET /api/admin/reports/rooms/usage
GET /api/admin/reports/payments

```

這些 API 可讓前端以統一方式與後端溝通，並由後端集中處理資料庫操作與權限驗證。

六、前端 Vue 頁面規劃

前端 Vue 可依照功能建立不同頁面與元件。

1. 頁面檔案規劃

```

LoginPage.vue
DashboardPage.vue
RoomListPage.vue
RoomDetailPage.vue
RoomAvailabilityPage.vue
CreateReservationPage.vue
MyReservationsPage.vue
UserListPage.vue
AfflicationManagePage.vue
ApprovalListPage.vue
PaymentListPage.vue
ReservationReportPage.vue
RoomManagePage.vue

```

2. 共用元件規劃

```

AuthenticatedLayout.vue
ApprovalActionButtons.vue
PaymentStatusBadge.vue

```

```
ReservationForm.vue
ReservationStatusBadge.vue
AvailabilitySlot.vue
```

共用元件可提升前端維護性。例如 `AuthenticatedLayout.vue` 可重複用於頁首與導覽列表；
`ReservationForm.vue` 可用於預約教室申請。

七、資料庫連結與資料操作技術

本系統使用 MySQL 儲存資料，Laravel 透過 `.env` 設定資料庫連線資訊，例如資料庫名稱、帳號、密碼與主機位置。

`.env` 設定範例如下：

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=database_project
DB_USERNAME=root
DB_PASSWORD=your_password
```

本系統資料庫部分以 DDL 指令匯入 MySQL 建立資料表，資料表結構依照 E-R Diagram 轉換而成，包含使用者、角色、單位、空間、時段、預約、付款與審核等資料。Laravel 後端透過 Model 操作已建立完成的資料表，並由 Controller 處理前端 Vue 傳來的請求，例如登入驗證、空間查詢、預約建立、衝突檢查、審核與付款狀態更新。透過此方式，可讓資料庫設計與系統功能維持一致，並確保主鍵、外鍵與資料關聯符合原本的 E-R Diagram 規劃。

例如建立預約時，後端會依序執行：

1. 接收前端傳來的 `room_id`、`date`、`time_slot_id`
2. 檢查使用者是否登入
3. 查詢使用者角色與所屬單位
4. 查詢 `room` 判斷是否可借用
5. 查詢 `reservation` 判斷是否有成功預約衝突
6. 若無衝突，建立 `reservation` 紀錄
7. 若需付款，建立 `payment` 紀錄
8. 回傳預約結果給前端

八、系統實作預期成果

完成後，本系統可提供校園空間租借的完整流程，包含空間查詢、預約申請、衝突檢查、審核、付款與紀錄查詢。使用者可透過網頁快速查詢可用空間並送出預約申請，行政人員可集中管理待審核預約與付款管理，系統管理員則可維護空間、單位、時段與價格資料。

透過資料庫的主鍵、外鍵與關聯表設計，系統能維持資料一致性，避免同一空間同一時段被重複預約。同時，透過角色與單位權限控管，也能確保不同使用者只能操作其被允許的功能與空間。

整體而言，本系統可改善傳統人工或分散式空間借用流程，提升校園空間管理效率，並提供後續統計與管理決策所需的資料基礎。

工作分配

功能	負責人
登入驗證、權限管理	余宥勝
空間與時段管理	張詠勝
預約流程	黃奕嘉
審核流程	劉育均
費用與付款管理	劉育均
查詢、紀錄、統計	余宥勝
測試	全組人員

Report01

1. 專題動機與目標

本專題旨在設計一套校園空間租借系統，協助學校有效管理教室、會議室、實驗室等之預約與使用情形

目前校園空間借用多以人工或分散系統處理，容易產生不同空間管理方式不一致，導致預約不到教室的問題。

因此，本系統希望透過資料庫整合空間、使用者、設備與預約資訊，提升管理效率與資料一致性。

本系統的主要目標如下：

1. 提供多種類型空間的統一管理與預約機制
2. 實現不同使用者之權限控管
3. 自動檢查預約時段衝突
4. 提供費用計算與使用統計功能

2. 系統使用者

1. 學生：
 - 可查詢空間資訊（教室容納人數、位置、設備...等）
 - 可預約一般教室或會議室
 2. 教授：
 - 可查詢空間資訊（教室容納人數、位置、設備...等）
 - 可預約一般教室、會議室、實驗室
 3. 行政人員：
 - 可查詢空間資訊（教室容納人數、位置、設備...等）
 - 可預約一般教室、會議室、實驗室
 - 負責審核預約申請、管理空間與設備
 - 處理費用相關事務
 4. 系統管理員：
 - 可查詢空間資訊（教室容納人數、位置、設備...等）
 - 負責系統整體設定，包括空間新增和刪除、權限設定與系統維護
-

3. 系統主要功能

1. 使用者資料管理：
 - 系統需記錄不同使用者的基本資料與角色，作為權限控管的依據
2. 空間資料管理：
 - 系統需管理不同類型空間（教室、會議室、實驗室）的基本資訊
3. 空間查詢功能：
 - 使用者可查詢各空間的基本資訊、可用時段與使用狀況
4. 空間預約功能：
 - 使用者可選擇空間與時間進行預約，系統需檢查是否符合條件
5. 預約審核功能：
 - 部分空間需經審核後才可使用，相關人員可進行審核與決策
6. 費用計算功能：
 - 系統可依空間與使用時間計算費用

7. 使用紀錄查詢：

- 系統需保留預約與使用紀錄，供後續查詢
-

4. 需求功能分析

4.1 功能性需求

1. 能建立與維護使用者資料
 2. 能建立與維護空間資料與空間類型
 3. 能提供空間查詢功能
 4. 能處理空間預約與取消預約
 5. 能檢查預約時段是否衝突
 6. 能進行預約審核
 7. 能計算空間使用費用
 8. 能保留預約與使用紀錄以供查詢
-

4.2 非功能性需求

1. 系統需保持資料一致性，避免重複預約或衝突情形
 2. 系統需具備支援多使用者同時查詢與預約
 3. 權限控管需明確，避免未授權使用者存取高階資源
 4. 系統應具備擴充性，以支援未來新增空間類型或設備
-

5. 資料分析

本系統主要管理以下資料對象：

1. 使用者：

記錄系統中所有使用者的基本資訊與角色，用於辨識身分與權限

2. 角色：

用來區分不同使用者的權限類型，例如學生、教授與管理員

3. 空間：

記錄各類型空間的基本資訊，例如名稱、容量、設備與所屬建築

4. 空間類型：
用來區分不同空間類別，例如教室、會議室或實驗室
 5. 預約紀錄：
用來描述使用者對空間的預約情況，包括時間與狀態
 6. 審核紀錄：
用來記錄預約是否通過審核及相關決策資訊
-

6. 資料關聯分析

1. 一位使用者可以進行多筆空間預約
 2. 一個空間可以被多位使用者在不同時間預約
 3. 使用者與空間之間透過預約紀錄形成關聯
 4. 使用者與角色之間為多對多關係
 5. 空間與空間類型之間為多對一關係
 6. 預約紀錄可能對應至多一筆審核紀錄
-

7. 運作規則

查詢規則

1. 顯示所有場地的資訊
2. 可依各種空間資訊進行查詢，包括空間類型、關鍵字或建築名稱

預約規則

1. 使用者只能預約其權限允許的空間類型
2. 預約時間不得重疊
3. 部分空間需經審核通過後才能使用（其他系所或是總務處所擁有場地）
4. 不需審核之空間以申請時間早晚作為預約優先順序
5. 預約成功後，同一空間在同一時間不可被重複預約
6. 使用者可取消尚未開始的預約
7. 取消預約，系統會立刻自動釋出時段
8. 預約狀態：已預約、空間

9. 借用時間最小單位為一小時

計費規則

1. 不論身份，租借總務處所擁有場地皆需收費
2. 費用依空間類型與使用時間計算
 - 使用時間 (hr) x 時段價格 (NTD / hr)
 - 不滿一小時，以一小時計算

審核規則

1. 審核過程：已登記 → 審核中 → 預約成功/預約失敗
2. 預約成功顯示查詢會顯示已預約外，其他查詢皆顯示為空閒

小補充

此系統的設備會跟場地視為一體，不需額外借用設備。只是作為場地的資訊。
舉例：成功預約 b1 meeting room 後，該場地之設備可自由使用。